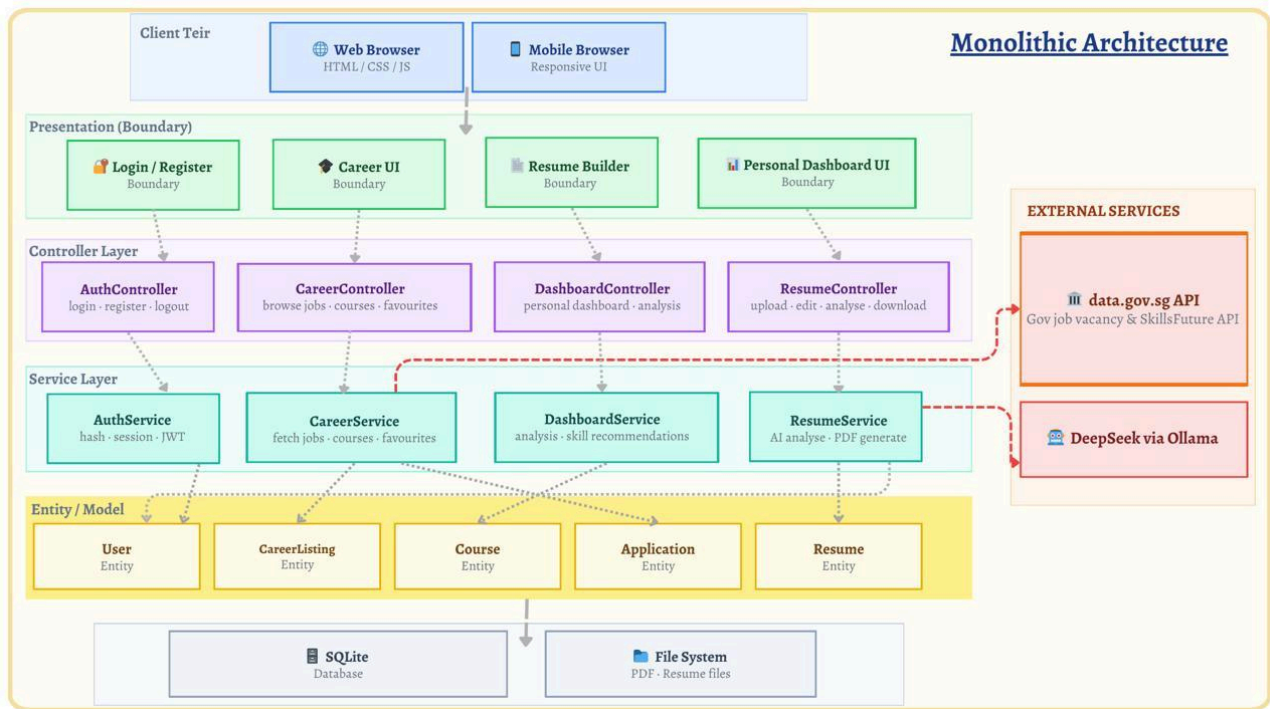


JobVago

Lab3 Submission: System Architecture

1. Overview Diagram



2. Architectural Style

Chosen Pattern: Monolithic Layered Architecture (MVC + Service Layer)

JobVago follows a Layered Architecture based on the Model-View-Controller (MVC) pattern, with the addition of a dedicated Service Layer sitting between the Controller and Model layers. This gives the system four distinct layers:

Layer	Responsibility
Presentation (View)	Renders the user interface using HTML templates. Receives data from controllers and displays it to the user. Contains no logic.
Controller	Handles incoming HTTP requests, calls the appropriate service, and returns a response (rendered page or redirect). Acts as the coordinator.
Service	Contains all business logic. The only layer that communicates with external systems (APIs, AI model). Keeps controllers thin.
Data Access (Model)	Manages all database read/write operations through an ORM. Represents persistent entity classes (User, CareerListing, Course, Application, Resume).

3. System Layers in Detail

3.1 Presentation Layer (Boundary Classes)

In the boundary-control-entity model, this layer contains the boundary classes that interact directly with the user. In JobVago, this is implemented using:

- `app/templates/`
- `app/static/`

This layer only renders data passed down from the controller. It performs no business logic, no database queries, and no API calls. The strict separation ensures that a change in the visual design would not require any changes to the logic layers.

3.2 Controller Layer (Control Classes)

The controller layer contains the control classes that coordinate use case flows. In Flask, controllers are implemented as Blueprints — a way to organise routes by functional domain. JobVago has four controllers:

Controller (Blueprint)	Use Cases Handled
AuthController	Register Account, Login, Logout
CareerController	Browse Industries, Browse Courses, Filter and Sort Courses, View Career Dashboard
DashboardController	View Personal Dashboard, View Favourite Courses, Manage Favourites (Add/Remove), View Past Analyses
ResumeController	Upload Resume, Manage Resume, Edit Resume, Analyse Resume (AI), View Analysis Result, View Skill Recommendations, Generate Edited PDF, Download Resume, Delete Resume

Each controller method (route handler) follows the same pattern: validate the HTTP request, call one or more service methods, and either render a template or redirect. Controllers are deliberately kept thin — they contain no business logic themselves.

3.3 Service Layer (Business Logic)

The service layer is not a standard part of basic MVC, but it is a widely used extension that improves the design. It sits between the controller and the data access layer and contains all business rules and application logic. Without a service layer, that logic would either get crowded into the controllers or scattered across model classes.

JobVago has four service classes:

Service Class	Responsibility
AuthService	Handles user registration, password hashing and verification, and session management via Flask-Login.
CareerService	Fetches and parses job vacancy and SkillsFuture course data from the data.gov.sg APIs. Also supports browsing, filtering, sorting, and favourites-related operations for careers and courses.
DashboardService	Prepares personalised dashboard data for the logged-in user, such as favourites, past resume analyses, and skill-gap recommendation results generated from resume analysis. It mainly coordinates and aggregates data for dashboard presentation.
ResumeService	Manages the full resume workflow, including upload, text extraction, AI-based analysis, grading, skill extraction, resume editing, PDF regeneration, and download support.

Some responsibilities inside CareerService and ResumeService may be implemented internally as helper functions or submodules. For example, resume analysis, grading, editing, and skill extraction are all part of ResumeService at the architectural level, even if they are separated internally in code for maintainability.

3.4 Data Access Layer (Entity Classes)

This layer contains the entity classes that represent persistent data. JobVago uses SQLite as the underlying database, accessed via SQLAlchemy as the ORM, which means database tables are defined as Python classes and SQL queries are generated automatically. The five main entity classes are:

- **User** — account info, hashed password, session fields
- **CareerListing** — job vacancy data fetched from data.gov.sg
- **Course** — SkillsFuture course data, linked to favourites
- **Application** — tracks a user's saved/applied jobs
- **Resume** — stores uploaded resume file path, analysis results, edit history

4. Addressing Key Design Issues (Bruegge Chapter 7.4)

4.1 Identifying and Storing Persistent Data

The system has two categories of persistent data, with a separate storage strategy for large files:

- **User account data:** Stored in the User entity class, persisted to the users table via SQLAlchemy. Includes hashed password, email, and session token.
- **User-generated content:** User-generated content is stored in entities such as Application and Resume, together with any associated favourite job or course records.
- **Resume files:** The actual PDF binary files are stored in the server's filesystem (in the uploads/ folder). Only the file path and metadata are stored in the database. This avoids storing large binary objects (BLOBs) in the database, keeping queries fast.

The `db.create_all()` call inside the application factory ensures that all tables are created from the ORM model definitions on startup, so no manual SQL schema setup is needed.

4.2 Providing Access Control

Access control and session security are implemented as follows:

- **Authentication (who are you?):** Handled by `AuthService` using Flask-Login. User passwords are never stored in plain text; they are hashed using Werkzeug's `generate_password_hash` before storage and verified with `check_password_hash` at login.
- **Authorisation (what can you do?):** All routes that require a logged-in user are decorated with `@login_required`. If an unauthenticated request reaches such a route, Flask-Login automatically redirects to the login page. Guest users can access the career browsing and industry browsing pages, but cannot access the personal dashboard or any resume features.
- **Session security:** Session cookies are configured with `HttpOnly=True` (prevents JavaScript from reading the cookie, reducing XSS risk) and `SameSite=Lax` (reduces CSRF risk). In production, `SESSION_COOKIE_SECURE=True` is set, so cookies are only sent over HTTPS.

5. External System Integrations

JobVago integrates with two external systems. Both are accessed exclusively through service classes in the Service Layer. No controller or model class communicates with them directly. This design decision ensures that if an external API changes, only the relevant service class needs to be updated.

External System	Purpose	Accessed By
data.gov.sg Jobs & SkillsFuture API	Provides government job vacancy listings and SkillsFuture course data used for browsing, filtering, and recommendation support.	CareerService
DeepSeek-R1 via Ollama (local LLM)	Powers AI resume analysis, grading, editing, and skill extraction. The model runs locally on the host machine, so no resume data is sent to a third-party cloud service.	ResumeService